Library Management System — Review III Report

Student: Suyash Singh
Register number: 24BYB0078
Course: BCSE302L / BCSE302P Database Systems Lab
Slot / **faculty:** G1 / Ramkumar T
Institution: Vellore Institute of Technology
Project type: Individual project

1. Introduction and problem statement

A library must know which books and copies it owns, who has borrowed them, when they are due, and which late fees remain unpaid. Manual registers can duplicate members, issue unavailable books, miss overdue returns, and miscalculate fines. This project keeps those records in a relational Oracle database and connects them to an interactive librarian dashboard.

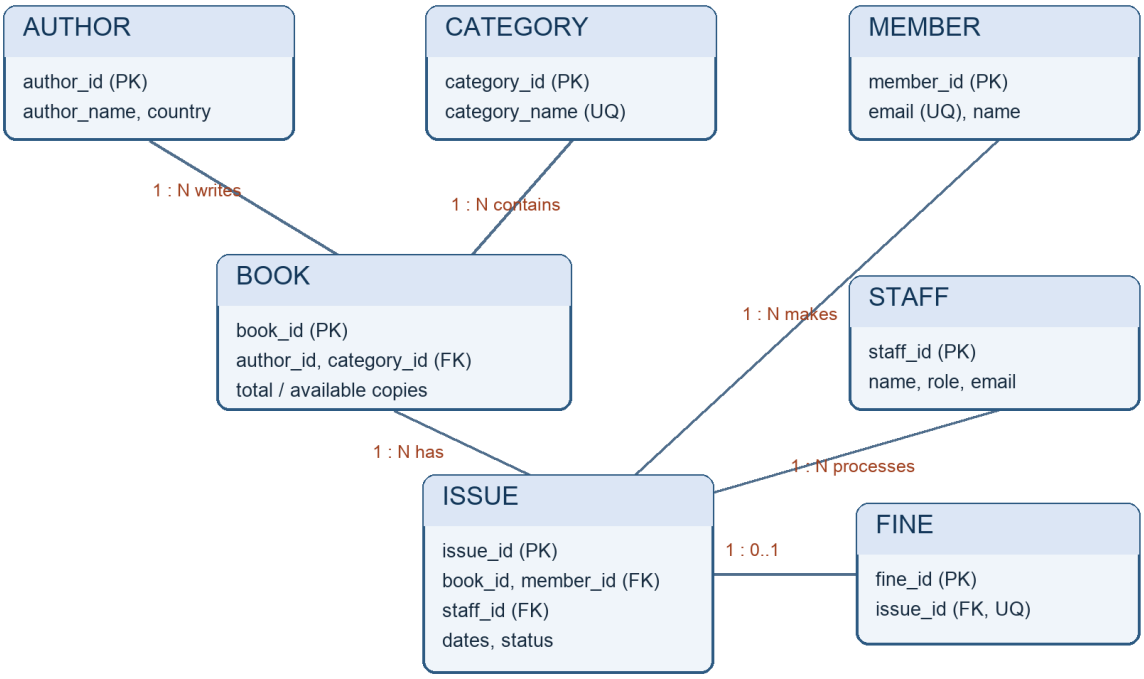
Review I defined the entities, relationships, and functional requirements. Review II created the tables, constraints, sample data, 15 queries, two procedures, two functions, an explicit cursor demonstration, and two triggers. Review III connects that database to a Flask API and browser interface. The browser interface extends the original Review I scope for the final integrated demonstration.

2. Functional requirements and completed system

Requirement	Implementation
Store books, authors, categories, members and staff	CRUD pages and Oracle tables `BOOK`, `AUTHOR`, `CATEGORY`, `MEMBER`, `STAFF`
Issue only available books	`issue_book` inserts an `ISSUE`; `trg_issue_insert` checks and decrements copies
Record due dates and returns	`ISSUE` dates/status; `return_book` procedure
Restore stock on return	`trg_issue_return` increments available copies on `Issued` to `Returned`
Calculate and track fines	`overdue_days`, `calculate_fine`, `FINE` table, payment-status update
Show activity and reports	Dashboard, issue history, overdue cursor page, 12 SQL-backed reports and CSV export
Demonstrate INSERT, UPDATE and DELETE	Member/book/author/category forms, fine payment, and deletion with FK protection

The application is a local librarian/admin demonstration. It has no member login, online payment, reservation or e-book feature.

3. ER diagram and relational schema



ER diagram showing AUTHOR and CATEGORY linked to BOOK, BOOK, MEMBER and STAFF linked to ISSUE, and the optional FINE for an ISSUE

`ISSUE` resolves the many-to-many relationship between books and members over time. `FINE.issue_id` is both a foreign key and unique, so an issue has at most one fine.

Relational schema:

```
AUTHOR(author_id PK, author_name, country)
CATEGORY(category_id PK, category_name UNIQUE)
BOOK(book_id PK, title, author_id FK→AUTHOR, category_id FK→CATEGORY,
     price, total_copies, available_copies)
MEMBER(member_id PK, name, email UNIQUE, phone, join_date, membership_type)
STAFF(staff_id PK, name, role, email UNIQUE)
ISSUE(issue_id PK, book_id FK→BOOK, member_id FK→MEMBER, staff_id FK→STAFF,
      issue_date, due_date, return_date, status)
FINE(fine_id PK, issue_id UNIQUE FK→ISSUE, amount, paid_status)
```

The exact Oracle DDL and constraints are in [sql/schema.sql](). They include primary and foreign keys, `NOT NULL`, `UNIQUE`, `CHECK`, and `DEFAULT`. The sample data in [sql/sample_data.sql]() has at least ten rows in every table: 10 authors, 10 categories, 10 books, 10 members, 10 staff, 15 issues, and 10 fines.

4. Database implementation and connectivity

```
Browser (HTML/CSS/JavaScript)
  → fetch /api/* JSON
Flask routes (validation, transactions, serialization)
  → python-oracledb connection pool
Oracle tables + SQL queries + PL/SQL procedures/functions/triggers
  → JSON results and dashboard/report output
```

The connection parameters are read from `.env` by [db.py](). Flask routes in [app.py]() use bound SQL parameters. Writes commit after dependent database work succeeds and roll back on Oracle errors. The issue and return routes call the Review II procedures; the application never manually adjusts stock for those

operations.

For a fresh database, run `sql/schema.sql`, then `sql/sample_data.sql`, then `sql/procedures.sql`. The sample inserts historical issues directly, so the triggers are installed **after** the sample data to avoid changing its prepared copy counts. Existing Review II databases should be verified instead of recreated. Run `python scripts/verify_oracle.py` after configuring credentials. A passing `/api/status` must show `connected: true` for the final Oracle demonstration. Demo mode uses static data and does not prove database integration.

5. SQL queries and outputs

The 15 Review II SQL queries are in [sql/queries.sql](). They cover joins, availability filtering, history, grouping, `LEFT JOIN`, `NOT EXISTS`, subqueries, `HAVING`, overdue dates, fine totals, and staff activity. The integrated Reports page offers 12 related live queries through `/api/reports/<name>`.

The following are **expected results from the provided sample data**, not captured output from a live Oracle instance:

Query / report	Expected sample result
Dashboard titles / copies / available	10 titles, 43 total copies, 38 available copies
Active and overdue issues (after 19 Sep 2026)	5 active issues, IDs 11–15; all 5 are overdue
Fine summary	Paid: ₹55 from 4 fines; Unpaid: ₹110 from 6 fines
Books by category	Fiction: 3; Classic: 2; Fantasy, Biography, History, Science, Mystery: 1 each; Self-Help, Technology, Education: 0
Staff activity	Staff IDs 1–5 handled 2 issues each; IDs 6–10 handled 1 each
Borrow count per member	Members 2–6 have 2 issues each; members 1 and 7–10 have 1 each
Inactive members	None in the sample

The live result depends on the Oracle server date and any changes made during the demonstration. Before submission, capture screenshots or SQL Developer output from your actual Oracle instance for this section and the five transactions below. The CSV button exports the rows currently returned by each report endpoint.

6. Procedures, functions, cursor and triggers

All Review II PL/SQL definitions are in [sql/procedures.sql]().

Object	Role in an integrated operation
`issue_book` procedure	Inserts a new borrowing transaction
`return_book` procedure	Locks the issue, records the return and inserts a positive fine
`overdue_days` function	Calculates nonnegative late calendar days
`calculate_fine` function	Multiplies late days by ₹5
`trg_issue_insert`	Rejects unavailable books and decrements stock after issue insert
`trg_issue_return`	Restores one copy when status changes from `Issued` to `Returned`
`overdue_cursor`	Review II explicit cursor prints each overdue issue in SQL Developer

The Database Logic page runs the same overdue selection through a PL/SQL `SYS_REFCURSOR` and fetches its rows into the UI. This is an actual cursor invocation; the Review II block remains available for a SQL Developer demonstration.

7. Five real-world transaction demonstrations

Use the detailed steps in docs/demo-checklist.md. Record the generated IDs during the live run, because `MAX(id)+1` may differ from the sample.

Transaction 1 — Register a member (INSERT)

User action: Members → Register Member. Enter a unique email, name, phone and membership type.
Database operation: `INSERT INTO MEMBER` with `join_date DEFAULT SYSDATE`; unique email and membership type constraints apply.
Result: A new member appears in the member list. Verify with `SELECT * FROM MEMBER WHERE email = '<email>';`.

Transaction 2 — Issue an available book (procedure + trigger)

User action: Issue Book → select member, book, staff and due date.
Database operation: Flask calls `issue_book`; it inserts `ISSUE`. `trg_issue_insert` validates available stock and decrements `BOOK.available_copies`.
Result: New `Issued` row; availability decreases by one. Verify the new `ISSUE` row and selected `BOOK.available_copies`.

Transaction 3 — Return an overdue book (procedure + functions + trigger)

User action: Return Book → select an overdue active issue → Return.
Database operation: `return_book` locks the issue, calls `calculate_fine` and `overdue_days`, updates its return date and status, and inserts a fine if positive. `trg_issue_return` restores one available copy.
Result: `ISSUE.status = 'Returned'`; one more available copy; one unpaid `FINE` with ₹5 per late calendar day. Verify all three tables using the issue ID.

Transaction 4 — Mark a fine as paid (UPDATE)

User action: Fine Management → Mark Paid.
Database operation: `UPDATE FINE SET paid_status = 'Paid' WHERE fine_id = :fine_id`.
Result: The fine moves from unpaid to paid totals. This records payment status; no payment gateway is involved.

Transaction 5 — Find and export overdue issues (cursor + report)

User action: Database Logic → Run Overdue Cursor Demo, then Reports → Overdue Issues → Export CSV.
Database operation: The PL/SQL reference cursor reads `ISSUE`, `MEMBER` and `BOOK` rows with `status = 'Issued' AND TRUNC(due_date) < TRUNC(SYSDATE)`; the report runs a corresponding joined SQL query.
Result: A row per overdue issue, including member, book and due date. The CSV contains the displayed report rows.

Additional UPDATE and DELETE demonstrations are available through editing/deleting a member, book, author or category. Parent deletion is rejected when referenced child rows exist, demonstrating foreign key protection.

8. Verification, limits and conclusion

Local verification: `venv/bin/python -m unittest discover -s tests -v` passes the demo and mocked database-boundary tests. The tests cover date binding, commit/rollback, return/fine response fields, stock editing, and cursor fetch behavior. Live Oracle tests are skipped without credentials; therefore database object compilation, actual Oracle transactions, and screenshots still require a run against the student's Oracle instance.

The system integrates the seven-table database with the Review II SQL and PL/SQL logic and presents five meaningful workflows through the UI. The main limits are local-only use without authentication, nonpersistent demo-mode writes, and `MAX(id)+1` ID allocation under concurrent requests. For the final viva, demonstrate with Oracle connected, run the verification script, and capture live transaction/report output.